Contents

ALGORITHM EXPLANATION AND CODE TRANSFORMATION.....4

User Input Handling.....4
Finding the Minimum Value.....5
Displaying the Minimum Value Inside the Circle.....6
Error Handling and Program Flow.....6
Complete Program Flow.....7
Output.....7
Advantages of the Program Design.....8

CONCLUSION.....9

REFERENCES.....10

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people's texts I clearly declare it through the good use of references following academic ethics.

(In the case that it is proved that part of the essay does not constitute an original work but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters):

HAYRÜNİSA DEMİRKALA

.....

Date: 22.10.2024

ALGORITHM EXPLANATION AND CODE TRANSFORMATION

This Python program makes the user input three numbers, and the system identifies the smallest from the three and the answer appears in a circle with a particular color using turtle graphics. The problem is divided into several key components: accepting the input from the user, comparing the numbers, presenting the result in a two-dimensional circle diagram, and there is input validation present in case a user inputs a wrong number.

First, it acquires three parameters through the input() method and then converts them to float for integer and decimal type. It subsequently employs the min() function, standard in Python, that quickly finds the variance of the three inputs. To see the result in a graphical format, the turtle module is used where a circle is drawn that is a light blue colour but includes a black border. The minimum value is shown in the centre of the circular graph, and if possible, it is rounded to an integer.

To deal with such results, the code also has a try-except block and if any inputs that violate these conditions make the program crash, it will encourage the user to re-input the correct ones. Dealing with input handling, the program logic and the visualization layer as individual modules simplifies the program architecture and makes it easy to manage. This ap

User Input Handling

First, the three numbers are taken from the user in the main method of the program. Here in this program we use Python's inbuilt function input() to take three numerical values (X, Y, Z) from the user. An additional step is then applied to convert the input values to floating point numbers which can be done by using the float() function. First, this results in adaptability in the program to accept both integers and decimals where necessary so that they are calculated correctly.

For example, if the user enters the whole number 5 the program considers it as 5.0 in order to retain consistency. It is also easy to perform the next operation of comparison after the conversion of float too. Likewise, the conversion process is enveloped in the try-catch block to handle erroneous values, for instance non numeric one. If the user has input the wrong data, the program throws ValueError and then prints the message to input the proper numbers. This mechanism helps avoid a program crash and makes it more user-friendly (Pal, 2022).

```python
# Main program logic
if __name__ == "__main__":
    try:
        # Get user input for the three numbers
        x = float(input("Enter the first number (X): "))
        y = float(input("Enter the second number (Y): "))
        z = float(input("Enter the third number (Z): "))

        # Find the minimum number
        minimum = find_minimum(x, y, z)
```

Finding the Minimum Value

The second step involves coming up with the least of the three numbers from the input that has just been obtained. This is done using min() functionality supported in python's programming language. The min() function actually eases the comparison logic, in which the function uses the three parameters and returns the lowest of all. Such a mechanism works better than writing conditional statements, hence making the code less obtrusive and more efficient (Ramos, 2024).

But to keep modularity the comparison logic is confined to one function named Find Minimum(). This function requires three arguments and finally calculates the minimum value by using the min() function.

```python
def display_on_circle(result):
    # Set up the turtle screen
    screen = turtle.Screen()
    screen.title("Minimum Number Display")

    # Create the turtle object for drawing the circle
    pen = turtle.Turtle()
    pen.hideturtle()
    pen.speed(1)

    # Draw a circle with a black boundary and light blue fill
    pen.penup()
    pen.goto(0, -100)
    pen.pendown()
    pen.color("black", "lightblue")
    pen.begin_fill()
    pen.circle(100)
    pen.end_fill()

    # Create another turtle object to display text
    text_pen = turtle.Turtle()
    text_pen.hideturtle()
    text_pen.penup()
    text_pen.goto(0, -10)
```

This function is invoked in the main program after the user enters the numbers so that the minimum value is found and then saved for further use.

Displaying the Minimum Value Inside the Circle

After the minimum value is found, using the turtle graphics module, a coloured circle is painted, and the result is written inside the circle. Turtle graphics also offer a simple and graphical window to display the results. The circle is outlined in black and filled with light blue, which gives a nice-looking output on the console.

The starting point is drawing the turtle screen tu and setting a title. The circle is drawn next with the pen turtle object, which has the boundary colour black and fills the circle with light blue. The circle is positioned in a way that the turtle's position points at the centre of the screen. The filling of the operation is done with the help of begin_fill and end_fill methods and ensures that the circle drawn is filled with the colour needed (Python.org, 2019).

```
# Write the result as an integer if it has no decimal part, otherwise as entered
display_value = int(result) if result.is_integer() else result
text_pen.write(f"Min: {display_value}", align="center", font=("Arial", 20, "bold"))
```

It provides a cleaner format of the output, thus providing the result that is expected from the user. The text is generated using the write() function, which also lets us point out the correct alignment, font type, etc.

Error Handling and Program Flow

The entire input and processing are enclosed within a try / except block so as to provide user-friendly handling of wrong inputs. If the user enters any other characters apart from numbers, an error message is displayed showing ValueError. It also helps in preventing program crashes at random and provides some feedback to the user of the program (Ruth, 2023).

```
try:
    # Get user input for the three numbers
    x = float(input("Enter the first number (X): "))
    y = float(input("Enter the second number (Y): "))
    z = float(input("Enter the third number (Z): "))

    # Find the minimum number
    minimum = find_minimum(x, y, z)

    # Display the result on a circle
    display_on_circle(minimum)

except ValueError:
    print("Please enter valid numbers!")
```

The program closes with turtle.done(), which maintains the turtle window open until it is manually closed. This puts the user in a position to view the result before the program terminates to avoid being terminated immediately after the program has processed the result.

Complete Program Flow

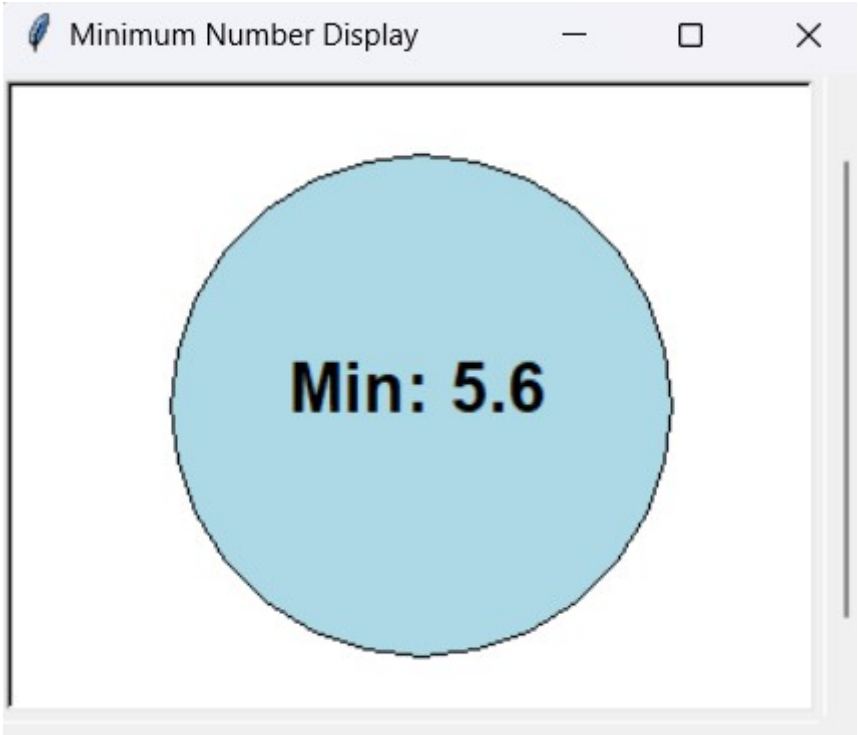
- The user is prompted to enter three numbers.
- The program finds the minimum among the three using the min() function.
- A circle with a light blue fill and black boundary is drawn using turtle graphics.
- The minimum value is displayed inside the circle and properly formatted.
- If invalid input is provided, an error message is shown, and the program prompts the user to enter valid numbers.

Output

After executing the program, we need to enter three numbers in sequence. For this program to find output, we enter three numbers: 33,23,5.6.

```
Enter the first number (X): 33
Enter the second number (Y): 23
Enter the third number (Z): 5.6
```

After entering the three numbers, the output is showing as follows:



The provided output shows the correct functionality of the Python program. After entering three numbers (33, 23, and 5.6), the program successfully identifies the smallest value, 5.6, and displays it inside a light blue circle with a black boundary using turtle graphics. The output confirms that the program correctly handles both integer and floating-point inputs and formats the result accurately, ensuring a smooth user experience with proper visualization.

Advantages of the Program Design

The following is the merit of this program: First, it is quite straightforward and easy since you just have to apply the min() function of Python language for comparison. This makes the output to be attractive and easily understood by the users of turtle graphics. Also, for stronger reliability, the program uses a try-except block for graceful exit in case of invalid input. All functionalities such as input, comparison and visualization that are coded can be easily maintained or added to if need arises due to its modularity.

CONCLUSION

This program exemplifies how all essential concepts of programming are incorporated within this simple solution including input management, control statements, and graphical output all enacted in Pygame Python setting. It offers an effective strategy of getting a solution to a problem by mixing the analytical method with the aesthetic sense. The program enters the user three numbers and using the min() function finds the smallest of them, and then showing the result inside the circle created with the turtle graphic. Therefore, through the process of modularity all of the components of the program are comprehensible, easy to navigate, and open to augmentation.

One must also note the quirky idea of incorporating turtle graphics into the program which makes the solution unique in its functionality. Not only does this make the output more interesting but it can also show how Python can be used in more than text-based programming. The code remains quite coherent as a whole and at the same time, the input, the comparison logic, the visualization is separated into functions or elements.

It also consists of a try-except block into the program to increase efficient error trapping. This feature makes the program robust and graceful where, if it finds that the inputs entered are invalid, it gives appropriate feedback. Which make the solution reliable and able to accommodate real life problems that may have instances of user involvement.

The overall result of this work is that Python can not only be used for logical computations but also for the unique graphic values. This program illustrates how a large problem can best be formulated and solved by devising it into smaller units with the help of modular programming and computer graphics.

REFERENCES

Nethala, A. (2017). *Python Program to find the Biggest and Smallest of 3 numbers*. [online] Medium. Available at: https://medium.com/programminginpython-com/python-program-to-find-the-biggest-and-smallest-of-3-numbers-5978c1e8fbfa [Accessed 22 Oct. 2024].

Pal, S. (2022). *Take Multiple Inputs From Users In One Line Using Python*. [online] Medium. Available at: https://geekpython.medium.com/take-multiple-inputs-from-users-in-one-line-using-python-7c7594237f5a [Accessed 22 Oct. 2024].

python.org (2019). *turtle — Turtle graphics — Python 3.7.4 documentation*. [online] Python.org. Available at: https://docs.python.org/3/library/turtle.html [Accessed 22 Oct. 2024].

Ramos , L.P. (2024). *Python's min() and max(): Find Smallest and Largest Values – Real Python*. [online] realpython.com. Available at: https://realpython.com/python-min-and-max/ [Accessed 22 Oct. 2024].

Ruth, E. (2023). *Python Errors and Exceptions: A Comprehensive Guide to Handling Errors and Exceptions in Python*. [online] Medium. Available at: https://medium.com/@ewho.ruth2014/python-errors-and-exceptions-a-comprehensive-guide-to-handling-errors-and-exceptions-in-python-3e6bf9381f71 [Accessed 22 Oct. 2024].